

Getting Started with React

Learn the basics of building dynamic web applications



What is React?

Definition: open-source JavaScript library for building user interfaces, particularly for single-page applications

Key Points:

- Developed by Facebook
- Component-based architecture
- Used for building user interfaces
- Efficient and flexible

Key Features of React

Virtual DOM: Faster updates by using a virtual representation of the UI.

JSX Syntax: Write HTML within JavaScript for a more readable code.

One-way Data Binding: Unidirectional data flow for predictable state management.

Component-based Structure: Reusable and isolated components.

Declarative UI: Describe what the UI should look like, and React will manage the rendering.

Component Lifecycle Methods: Methods that run at specific points in a component's life to handle events and state changes.

State Management: Manage and maintain the state of the application efficiently within components or using state management libraries like Redux.

Hooks: Use state and other React features without writing a class.

Context API: Share data across components without passing props manually at every level.

Server-side Rendering (SSR): Improve performance and SEO by rendering components on the server before sending them to the client.

Setting Up React

1. Install Node.js and npm:

- Download and install Node.js from the official website: nodejs.org
- npm (Node Package Manager) comes bundled with Node.js

2. Create a New React App:

- Open your terminal or command prompt
- Run the command: `npx create-react-app my-app`
 - `npx` is a package runner tool that comes with npm 5.2+
 - `create-react-app` is a React app generator
 - `my-app` is the name of your project

3. Navigate to the App Directory:

- Change directory to your new project folder
- Run the command: `cd my-app`

4. Start the Development Server:

- Start the React development server
- Run the command: `npm start`
 - This will start the development server and open your React app in the default web browser
 - The development server runs on `http://localhost:3000` by default

5. Explore the Project Structure:

- Open your project in a code editor (e.g., VSCode)
- Understand the basic structure:
 - `public/`: Contains the public assets like `index.html`
 - `src/`: Contains the source code for your React components
 - `node_modules/`: Contains all the project dependencies

6. Modify the Initial React App:

- Open `src/App.js` and make changes to see live updates
- Use `src/index.js` to include new components

Understanding JSX

JSX stands for JavaScript XML: A syntax extension for JavaScript.

Allows writing HTML in React: Embeds HTML directly within JavaScript code.

Transpiles to JavaScript: Babel converts JSX into standard JavaScript.

Makes code more readable and easier to write: Simplifies the creation of complex UI elements.

```
// JSX in action
import React from 'react';

function Greeting() {
  return (
    <div className="greeting">
      <h1>Hello, world!</h1>
      <p>This is a JSX example.</p>
    </div>
  );
}

export default Greeting;
```

Annotations:

`import React from 'react';`: Imports the React library.

`function Greeting() { ... }`: Defines a functional component.

`return ( ... )`: Uses JSX to return a React element.

`<div className="greeting"> ... </div>`: Embeds HTML within JavaScript.

`<h1>` and `<p>`: HTML elements written in JSX.

Components in React

Functional Components:

- Simplest way to create a component.
- Defined as JavaScript functions.

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

Class Components:

- ES6 classes that extend `React.Component`.
- Useful for managing state and lifecycle methods.

Example:

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}</h1>;  
  }  
}
```

Props and State:

- **Props:** Used to pass data to components.
- **State:** Manages dynamic data within a component.

Example using state in a class component:

```
class Clock extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { date: new Date() };  
  }  
  
  render() {  
    return (  
      <div>  
        <h1>It is {this.state.date.toLocaleTimeString()}.</h1>  
      </div>  
    );  
  }  
}
```

Lifecycle Methods:

- Methods that run at specific points in a component's life.
- Common methods: `componentDidMount`, `componentDidUpdate`, `componentWillUnmount`.

Props and State

Props:

- Short for properties, used to pass data between components.
- Immutable and read-only.
- Passed from parent component to child component.

Example:

```
function Greeting(props) {  
  return <h1>Hello, {props.name}</h1>;  
}  
// Usage  
<Greeting name="Alice" />
```

State:

- Manages dynamic data within a component.
- Mutable and can be changed using setState (in class components) or useState (in functional components).
- Local to the component and can influence its rendering.

Example using state in a class component:

```
class Clock extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { date: new Date() };  
  }  
  
  componentDidMount() {  
    this.timerID = setInterval(() => this.tick(), 1000);  
  }  
  
  componentWillUnmount() {  
    clearInterval(this.timerID);  
  }  
  
  tick() {  
    this.setState({  
      date: new Date()  
    });  
  }  
  
  render() {  
    return (  
      <div>  
        <h1>It is {this.state.date.toLocaleTimeString()}.</h1>  
      </div>  
    );  
  }  
}
```

Example using state in a functional component:

```
function Clock() {  
  const [date, setDate] = React.useState(new Date());  
  
  React.useEffect(() => {  
    const timerID = setInterval(() => setDate(new Date()), 1000);  
    return () => clearInterval(timerID);  
  }, []);  
  
  return (  
    <div>  
      <h1>It is {date.toLocaleTimeString()}.</h1>  
    </div>  
  );  
}
```

Handling Events

1. Adding Event Handlers in React:

- Events in React are named using camelCase, rather than lowercase.
- You pass a function as the event handler rather than a string.

2. Using Synthetic Events:

- React's events are normalized across different browsers, meaning you don't need to worry about compatibility issues.
- These synthetic events are instances of `SyntheticEvent`.

3. Common Events:

- **onClick:** Triggered when an element is clicked.

```
function ActionButton() {  
  function handleClick() {  
    alert('Button clicked!');  
  }  
  
  return (  
    <button onClick={handleClick}>  
      Click Me  
    </button>  
  );  
}
```

- **onChange:** Triggered when the value of an input element changes.

```
class NameForm extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { value: '' };  
  
    this.handleChange = this.handleChange.bind(this);  
    this.handleSubmit = this.handleSubmit.bind(this);  
  }  
  
  handleChange(event) {  
    this.setState({ value: event.target.value });  
  }  
  
  handleSubmit(event) {  
    alert('A name was submitted: ' + this.state.value);  
    event.preventDefault();  
  }  
}
```



```

render() {
  return (
    <form onSubmit={this.handleSubmit}>
      <label>
        Name:
        <input type="text" value={this.state.value} onChange={this.handleChange} />
      </label>
      <button type="submit">Submit</button>
    </form>
  );
}
}

```

- **onSubmit:** Triggered when a form is submitted.

```

function LoginForm() {
  const handleSubmit = (event) => {
    event.preventDefault();
    alert('Form submitted!');
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Username:
        <input type="text" name="username" />
      </label>
      <label>
        Password:
        <input type="password" name="password" />
      </label>
      <button type="submit">Login</button>
    </form>
  );
}

```

Conditional Rendering

1. Rendering Different Components or Elements Based on a Condition:

- Conditional rendering in React works the same way conditions work in JavaScript.
- You can use JavaScript operators like `if`, `else`, and ternary operators to decide which elements to render.

2. Using `if` Statements:

Example:

```
function Greeting(props) {  
  const isLoggedIn = props.isLoggedIn;  
  if (isLoggedIn) {  
    return <h1>Welcome back!</h1>;  
  }  
  return <h1>Please sign up.</h1>;  
}
```

3. Using Ternary Operators:

Example:

```
function Greeting(props) {  
  return (  
    <div>  
      {props.isLoggedIn ? (  
        <h1>Welcome back!</h1>  
      ) : (  
        <h1>Please sign up.</h1>  
      )}  
    </div>  
  );  
}
```

4. Using Logical `&&` Operator:

Example:

```
function Mailbox(props) {  
  const unreadMessages = props.unreadMessages;  
  return (  
    <div>  
      <h1>Hello!</h1>  
      {unreadMessages.length > 0 && (  
        <h2>  
          You have {unreadMessages.length} unread messages.  
        </h2>  
      )}  
    </div>  
  );  
}
```

5. Inline **if-else** with Conditional Operator:

Example:

```
function UserGreeting() {  
  return <h1>Welcome back!</h1>;  
}  
  
function GuestGreeting() {  
  return <h1>Please sign up.</h1>;  
}  
  
function Greeting(props) {  
  const isLoggedIn = props.isLoggedIn;  
  return (  
    isLoggedIn ? <UserGreeting /> : <GuestGreeting />  
  );  
}
```

Lists and Keys

1. Rendering Lists of Data Using the `map()` Function:

- In React, you can build collections of elements and include them in JSX using curly braces `{}`.

Example:

```
const numbers = [1, 2, 3, 4, 5];
const listItems = numbers.map((number) =>
  <li key={number.toString()}>
    {number}
  </li>
);

function NumberList() {
  return (
    <ul>
      {listItems}
    </ul>
  );
}
```

2. Importance of Keys for List Items:

- Keys help React identify which items have changed, been added, or removed.
- Keys should be given to the elements inside the array to give the elements a stable identity.
- Best practice is to use a string that uniquely identifies a list item among its siblings.

3. Keys Help Identify Which Items Have Changed:

- React uses keys to know which elements to update, preventing unnecessary re-renders.

Example of using keys correctly:

```
function TodoList(props) {
  const todos = props.todos;
  const todoItems = todos.map((todo) =>
    <li key={todo.id}>
      {todo.text}
    </li>
  );
  return (
    <ul>
      {todoItems}
    </ul>
  );
}
```

Forms In React

1. Controlled Components:

- Form elements whose values are controlled by React state.
- Every state mutation will have an associated handler function.

Example:

```
class NameForm extends React.Component {
  constructor(props) {
    super(props);
    this.state = { value: '' };

    this.handleChange = this.handleChange.bind(this);
    this.handleSubmit = this.handleSubmit.bind(this);
  }

  handleChange(event) {
    this.setState({ value: event.target.value });
  }

  handleSubmit(event) {
    alert('A name was submitted: ' + this.state.value);
    event.preventDefault();
  }

  render() {
    return (
      <form onSubmit={this.handleSubmit}>
        <label>
          Name:
          <input type="text" value={this.state.value} onChange={this.handleChange} />
        </label>
        <button type="submit">Submit</button>
      </form>
    );
  }
}
```

2. Handling Form Submissions:

- Form submission is typically handled by preventing the default form action and processing the form data using JavaScript.

Example:

```
function LoginForm() {
  const [username, setUsername] = React.useState('');
  const [password, setPassword] = React.useState('');

  const handleSubmit = (event) => {
    event.preventDefault();
    alert(`Username: ${username}, Password: ${password}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Username:
        <input type="text" value={username} onChange={(e) =>
setUsername(e.target.value)} />
      </label>
      <label>
        Password:
        <input type="password" value={password} onChange={(e) =>
setPassword(e.target.value)} />
      </label>
      <button type="submit">Login</button>
    </form>
  );
}
```

3. Managing Form State:

- State is typically managed using the `useState` hook for functional components or `this.state` for class components.

Example:

```
function SignupForm() {
  const [email, setEmail] = React.useState('');
  const [password, setPassword] = React.useState('');

  const handleEmailChange = (event) => {
    setEmail(event.target.value);
  };

  const handlePasswordChange = (event) => {
    setPassword(event.target.value);
  };

  const handleSubmit = (event) => {
    event.preventDefault();
    alert(`Email: ${email}, Password: ${password}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Email:
        <input type="email" value={email} onChange={handleEmailChange} />
      </label>
      <label>
        Password:
        <input type="password" value={password} onChange={handlePasswordChange} />
      </label>
      <button type="submit">Sign Up</button>
    </form>
  );
}
```

React Hooks

1. **useState**: Managing State in Functional Components

- Allows you to add state to functional components.
- Returns a state variable and a function to update it.

Example:

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

2. **useEffect**: Handling Side Effects

- Allows you to perform side effects in functional components.
- Can replace lifecycle methods like `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`

Example:

```
import React, { useState, useEffect } from 'react';

function Timer() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const timer = setInterval(() => {
      setCount(prevCount => prevCount + 1);
    }, 1000);
    return () => clearInterval(timer);
  }, []);

  return <h1>{count}</h1>;
}
```


3. Custom Hooks

- Allows you to create reusable hooks.
- Can encapsulate common logic that can be reused across components.

Example:

```
import { useState, useEffect } from 'react';

function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return width;
}

function DisplayWidth() {
  const width = useWindowWidth();

  return <div>Window width: {width}</div>;
}
```